UML class diagram - services: who resolves, plans, executes, verifies and composes

«interface» SemanticSupplier
«impl» DbtSemanticLayer [295B]
precedence over inferred
+ definitions(): Binding[]

«supplies» [295B]

Binding
see Figure 3

«creates»

SemanticResolver
margin_threshold: float
retrieval: BM25 + embeddings
model: ModelClient
+ resolve(question): Binding[]
+ ask(term, candidates)
+ confirm(term, choice)

«invokes»

Plan
see Figure 3

«creates»

PlanSynthesiser
hop_ceiling: int
plan_cache: signature → Plan
+ synthesise(q, bindings): Plan
+ validate(plan): Errors
+ revise(plan, failure) [295B]

«invokes»

«uses»

External model provider
frontier: plan, SQL · small: prose

«HTTPS»

ModelClient
provider: organisation's own key
mode: FULL | SCHEMA_ONLY | FILTERED_SAMPLES
+ plan(prompt) frontier
+ sql(prompt) frontier
+ prose(prompt) small

«reasons via» prose

«reasons via»

«value» StepResult
title: str
sql_executed: SQL
attempts: int
rows_in / rows_out: int
grain: str
output_ref: hash
sample_rows

«returns»

«abstract» SpecialistAgent
«impl» FetchAgent
«impl» AnalyticsAgent
«impl» VisualizationAgent [295B]
«impl» MLAgent [295B]
loop: reason → act → observe
+ run(step): StepResult
+ repair(sql, error)

1..* — *dispatches* — 1

«acts via»

«receives»

Source
see Figure 3

1 — *accesses* — 1

«interface» SourceConnector
«impl» PostgresConnector
«impl» FileConnector (DuckDB)
«impl» RestJsonConnector [295B]
+ describe()
+ sample(n)
+ execute(sql)
+ profile(cols)

«writes»

AuditEvent
see Figure 3

Legend
- service class (Figure 1 component)
- CMPE 295B scope
- domain class from Figure 3
- dependency «invokes» «creates» «uses»
- association

Orchestrator
plan: Plan (approved) · state: RunState · budget / cost_spent · retry_cap: int
every StepResult returns here; agents never call each other
+ run(plan) · + dispatch(step): StepResult · + compose_answer(): Answer · + halt(reason) · + replan() [295B]

«invokes»

CrossSourceReconciler
sql via: SourceConnector
match: Jaccard + edit distance
+ profile_keys(a, b)
+ propose(a, b): MappingProposal
+ coverage(a, b)

«creates»

MappingProposal
see Figure 3

«invokes» per hop

HopVerifier
planner · renderer · guard · runner
sql via: SourceConnector
no model calls; 5 s timeout
+ verify_hop(ctx): HopVerdict
+ rerun_checks(hop): HopVerdict
+ doubt_signals(inv)

«produces»

HopVerdict + CheckResults
see Figure 3

«invokes» per hop

LineageRecorder
store: Application + Artifact
content-addressed outputs
+ record(step, result, checks): Hop
+ replay(hop): Hop
+ export(investigation)

«records»

Hop
see Figure 3

«creates»

Answer
see Figures 3, 5

«awaits»

PlanApproval
see Figure 3

Flow: resolve terms → synthesise plan → analyst approval → dispatch steps → StepResult → verify (SQL) → record Hop → compose_answer (small model via ModelClient) → Answer.

Checks never use a model: HopVerifier and CrossSourceReconciler issue independent SQL through SourceConnector. Answer text is written only from verified hops, each sentence citing its hop.